

Lecture 20: Bipartiteness, SAT, Reductions, and Vertex Cover

*Instructor: Sourav Chakraborty**Scribe: Soumyadip Mahesh, Sourav Kundu*

1 Bipartite Graph and SAT Formulation

Problem Given a graph:

$$G = (V, E), \quad V = \{v_1, v_2, \dots, v_n\}$$

We want: Determine whether G is bipartite or not.

Note: A graph is bipartite if and only if it has no odd cycles.

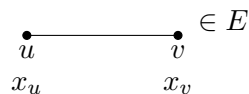
Exc:- How to find odd cycles in a graph?

SAT Reduction: Assign a Boolean variable x_u to each vertex $u \in V$. For every edge $(u, v) \in E$, enforce:

$$(x_u \oplus x_v)$$

This ensures adjacent vertices receive different colors (values). The equivalent **CNF (Conjunctive Normal Form)** for an edge is:

$$(x_u \vee x_v) \wedge (\bar{x}_u \vee \bar{x}_v)$$

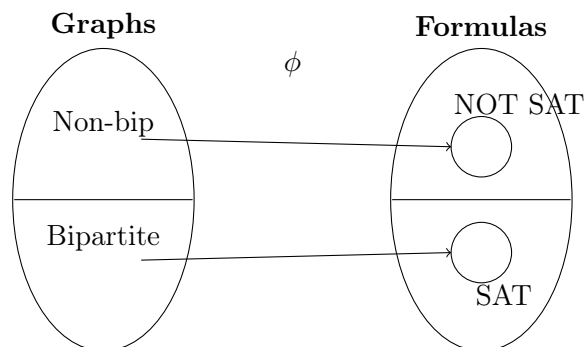


$$(x_u \oplus x_v)$$

$$(x_u \vee x_v) \wedge (\bar{x}_u \vee \bar{x}_v)$$

$$G \longrightarrow \phi(x_1, \dots, x_n)$$

$$G \longrightarrow \bigwedge_{(u,v) \in E} [(x_u \vee x_v) \wedge (\bar{x}_u \vee \bar{x}_v)] = \phi(G)$$



The complete SAT instance for the graph is:

$$\phi(G) = \bigwedge_{(u,v) \in E} [(x_u \vee x_v) \wedge (\bar{x}_u \vee \bar{x}_v)]$$

Theorem:

$$G \text{ is bipartite} \iff \phi(G) \text{ is satisfiable}$$

Therefore, **Bipartiteness** $\leq_p$ **2-SAT**

2 Polynomial-Time Reduction

Let $L_1 \in \Sigma_1^*$ and $L_2 \in \Sigma_2^*$ such that $L_1 \leq_p L_2$. Then there exists a function:

$$\rho : \Sigma_1^* \rightarrow \Sigma_2^*$$

Such that:

$$x \in L_1 \iff \rho(x) \in L_2$$

and ρ is computable in polynomial time.

Interpretation: If we can solve L_2 , we can solve L_1 . This process is called **reduction** and as ρ is computable in polynomial time, it is called **polynomial-time reduction**.

3 Why Reductions?

3.1 For solving new problems

- Reducing a problem into a known "easy" problem.
- Reducing a problem into a well-studied "hard" problem.

Heuristics

Algorithms without proof of correctness or proof of complexity.

- **Example:** Greedy algorithms.
- **Properties:** Fast, easy to implement, and works well on “typical” inputs.

Solvers

Common general-purpose solvers include:

- SAT Solvers
- ILP Solvers
- SMT (Satisfiability Modulo Theories) Solvers
- QBF (Quantified Boolean Formula) Solvers

Strategy: Reduce your specific problem to a well-studied hard problem, then use a specialized solver or heuristic to find a solution.

3.2 For proof of Hardness

Vertex Cover Problem

Problem: Given a graph G , find a vertex cover (a set of vertices such that every edge is incident to at least one vertex in the set). If a polynomial-time solution is unknown, we show it is as hard as another hard problem.

Solution ::

1. Introduction

intuition: Here we have to show that ,Vertex cover is NP complete. To do this , we must first show that VC belongs to the complexity class NP and then perform a polynomial-time reduction from a known NP-Complete problem—in this case, 3-SAT.

Vertex Cover (VC)

Input: An undirected graph $G = (V, E)$ and an integer k .

Question: Does there exist a subset $V' \subseteq V$ such that $|V'| \leq k$ and for every edge $(u, v) \in E$, at least one of u or v is in V' ?

3-Satisfiability (3-SAT)

Input: A Boolean formula ϕ in Conjunctive Normal Form (CNF) where each clause has exactly three literals.

Question: Is there a truth assignment to the variables that makes ϕ true?

2. VC is in NP

To prove VC is in NP, we provide a certificate: a subset of vertices V' .

- **Verification:** We can check in polynomial time if $|V'| \leq k$ and then iterate through every edge $(u, v) \in E$ to verify that either $u \in V'$ or $v \in V'$.

- Since verification takes $O(|V| + |E|)$ time, $VC \in NP$.

3. Reduction: 3-SAT $\leq_p$ Vertex Cover

Given a 3-SAT formula ϕ with n variables $\{x_1, x_2, \dots, x_n\}$ and m clauses $\{C_1, C_2, \dots, C_m\}$, we construct a graph G and an integer k such that ϕ is satisfiable if and only if G has a vertex cover of size $k = n + 2m$.

The Construction (Gadgets)

1. **Variable Gadgets:** For each variable x_i , create two nodes labeled x_i and $\neg x_i$ connected by an edge.
 - To cover this edge, any vertex cover must contain at least one of these two nodes.
2. **Clause Gadgets:** For each clause $C_j = (l_1 \vee l_2 \vee l_3)$, create a triangle (a K_3 clique) with three nodes, one for each literal.
 - To cover the three edges of the triangle, any vertex cover must contain at least two of these nodes.
3. **Communication Edges:** Connect each node in a clause gadget to the corresponding node in the variable gadget. For example, if $C_1 = (x_1 \vee \neg x_2 \vee x_3)$, connect the first node of the C_1 triangle to node x_1 in the variable gadget, the second to $\neg x_2$, and the third to x_3 .

4. Formal Proof of Correctness

We must show: ϕ is satisfiable $\iff G$ has a vertex cover of size $k = n + 2m$.

Direction 1: Completeness ($\implies$)

Suppose ϕ is satisfiable. We construct a vertex cover V' as follows:

- From Variable Gadgets: If x_i is TRUE, add node x_i to V' . If x_i is FALSE, add node $\neg x_i$. This contributes n vertices and covers all variable gadget edges.
- From Clause Gadgets: In each clause C_j , at least one literal must be TRUE (since ϕ is satisfied). Let that literal be l^* . The edge connecting the node for l^* in the clause triangle to its variable gadget counterpart is already covered by the variable node. We then pick the other two nodes of the triangle to be in V' . This contributes $2m$ vertices and covers all three internal edges of the triangle.
- Total Size: $|V'| = n + 2m = k$. All edges are covered.

Direction 2: Soundness ($\Leftarrow$)

Suppose G has a vertex cover V' of size $k = n + 2m$.

- To cover the n variable edges, V' must pick at least 1 node per edge (total $\geq n$).
- To cover the m triangles, V' must pick at least 2 nodes per triangle (total $\geq 2m$).
- Since $|V'| = n + 2m$, V' must pick exactly one node from each variable gadget and exactly two nodes from each clause gadget.
- **The Assignment:** If $x_i \in V'$, set $x_i = \text{TRUE}$. If $\neg x_i \in V'$, set $x_i = \text{FALSE}$.
- **Why it satisfies ϕ :** In any clause triangle C_j , only two nodes are in V' . This means the edge connecting the third node to the variable gadget must be covered by the variable node. Therefore, the literal corresponding to that third node must be TRUE in our assignment. Thus, every clause has at least one TRUE literal.

5. Complexity and Conclusion

- Time Complexity: The number of vertices is $2n + 3m$ and the number of edges is $n + 3m + 3m$. The construction is clearly polynomial in terms of the input size of ϕ .

- Conclusion: Since we have shown a polynomial-time reduction from 3-SAT to VC, and VC is in NP, the Vertex Cover problem is NP-complete.